

Faculte DES Sciences Ben M'SICK - CASABLANCA

Projet de Machine Learning

Detection des Maladies des Plantes par Deep Learning

Etudiantes :

Malak Boussetta & Nora Idouaouzal

Encadrant :

Pr. Habib Ben Lahmar

Co-Encadrante :

Salma Hanouni

Année Universitaire 2025-2026

Table des matieres

1	Contexte et Problématique	5
1.1	Contexte Général.....	5
1.2	Problématique Scientifique	5
1.3	Objectifs du Projet.....	5
2	Méthodologie	5
2.1	Approche Générale	5
2.2	Choix Technologiques.....	6
2.3	Stratégie de Transfer Learning	6
3	Collecte et Structuration des Données.....	7
3.1	Présentation du Dataset PlantVillage.....	7
3.2	Infrastructure de Stockage et Organisation	7
4	Exploration et Audit des Données (EDA).....	8
4.1	Audit Qualité du Dataset	8
4.2	Distribution des Classes.....	8
5	Prétraitement et Pipeline de Données.....	9
5.1	Construction des Pipelines tf.data	9
5.2	Augmentation des Données	10
5.3	Normalisation	10
6	Architecture du Modèle	10
6.1	EfficientNetB0 — Justification Scientifique.....	10
6.2	Architecture Complète du Réseau	10
6.3	Compilation du Modèle.....	11
7	Entraînement du Modèle	11
7.1	Callbacks de Supervision	11
7.2	Courbes de Convergence — Accuracy.....	11
7.3	Courbes de Convergence — Loss	12
8	Évaluation et Résultats	13
8.1	Métriques Globales	13
8.2	Courbe ROC et Aire sous la Courbe (AUC).....	14
8.3	Matrice de Confusion	15
8.4	Performances Détaillées par Classe	15
8.5	Analyse Volumétrique — Treemap Hybride.....	16
9	Perspectives de Déploiement.....	17
9.1	Application Web — Streamlit.....	17

9.2	Déploiement Mobile — TensorFlow Lite.....	18
10	Discussion et Conclusion	18
10.1	Analyse des Résultats	18
10.2	Points Forts du Projet.....	19
10.3	Limites Identifiées.....	19
10.4	Conclusion	19

Liste des figures

Figure 1 Distribution des images par classe dans le jeu de validation (38 classes). La ligne rouge indique la moyenne.	9
Figure 2 de l'accuracy en entraînement et en validation sur 20 époques. Palier final : 96,52 % (validation).	12
Figure 3 Évolution de la fonction de perte (cross-entropie) sur 20 époques. Convergence stable sans divergence train/val.	13
Figure 4 Tableau de bord des 4 métriques globales (EfficientNetB0). Chaque barre affiche le pourcentage de réussite.	14
Figure 5 Courbe ROC micro-moyenne sur 38 classes ($AUC \approx 1,00$). La diagonale pointillée représente le classificateur aléatoire.	14
Figure 6 Matrice de confusion 38×38 (heatmap Blues). La diagonale principale représente les prédictions correctes.....	15
Figure 7 Heatmap de performance par classe. Palette RdYlGn : vert = excellents scores ($F1 \geq 0,97$), orange = classes plus délicates.	16
Figure 8 Treemap hybride Volume × F1-Score (Top 15 classes). Taille = représentativité, Couleur. 17	

1 Contexte et Problématique

1.1 Contexte Général

Les maladies des plantes représentent l'une des principales menaces pour la sécurité alimentaire mondiale. Selon la FAO, elles sont responsables de pertes de rendements agricoles estimées entre 20 et 40 % chaque année, avec des impacts économiques et humanitaires considérables, notamment dans les pays en développement.

La majorité des agriculteurs, en particulier dans les régions rurales, n'a pas accès à une expertise agronomique spécialisée pour identifier rapidement et précisément les pathologies qui affectent leurs cultures. Un diagnostic tardif ou erroné entraîne une propagation accélérée des maladies, une utilisation inadaptée des traitements phytosanitaires et des pertes économiques irréversibles.

1.2 Problématique Scientifique

Comment concevoir un système d'intelligence artificielle capable de classier automatiquement 38 catégories de maladies végétales à partir d'une image de feuille, avec une précision supérieure à 95 %, tout en restant déployable sur des appareils mobiles ou en tant que service web ?

La reconnaissance de pathologies végétales par vision par ordinateur soulève plusieurs défis techniques :

- Variabilité intra-classe : une même maladie peut présenter des symptômes visuels très différents selon le stade d'évolution, la variété végétale ou les conditions d'éclairage.
- Similarité inter-classes : certaines pathologies partagent des signatures visuelles proches (taches, décolorations, nécroses), complexifiant la discrimination.
- Déséquilibre des classes : le nombre d'images disponibles par catégorie varie considérablement (275 à 5 357 dans PlantVillage).
- Généralisation hors-distribution : les images en conditions réelles (fond naturel, ombres) diffèrent des images de laboratoire du dataset.

1.3 Objectifs du Projet

- Construire un pipeline complet et reproductible de traitement des données image.
- Implémenter une stratégie de Transfer Learning via EfficientNetB0 pré-entraîné sur ImageNet.
- Atteindre une accuracy de validation ≥ 95 % sur les 38 classes de PlantVillage.
- Évaluer quantitativement et visuellement les performances par des métriques multi-dimensionnelles.
- Préparer les bases d'un déploiement applicatif via Streamlit ou TensorFlow Lite.

2 Méthodologie

2.1 Approche Générale

Le projet suit une méthodologie structurée en six phases séquentielles, inspirée du cadre CRISP-DM (Cross-Industry Standard Process for Data Mining) adapté aux projets de Deep Learning pour la vision par ordinateur :

Phase	Étape	Description
1	Collecte & Structuration	Acquisition du dataset PlantVillage, organisation de l'arborescence projet, persistance sur Google Drive
2	Exploration & Audit (EDA)	Analyse de la distribution des classes, détection des images corrompues, suppression des doublons MD5
3	Prétraitement & Pipeline	Construction des pipelines tf.data, normalisation, augmentation de données, chargeurs optimisés GPU
4	Modélisation	Transfer Learning EfficientNetB0, gel des poids, ajout tête de classification, compilation
5	Entraînement	Entraînement supervisé avec callbacks (ModelCheckpoint, EarlyStopping, ReduceLROnPlateau)
6	Évaluation & Visualisation	Métriques globales, matrice de confusion, courbe ROC, rapports par classe, livrables graphiques

2.2 Choix Technologiques

Composant	Technologie	Justification
Framework DL	TensorFlow 2.x / Keras	API haut niveau, optimisation GPU native, pipeline tf.data
Architecture	EfficientNetB0	Meilleur ratio performance/coût, 5,3M paramètres, compatible mobile
Pré-entraînement	ImageNet (1,2M images)	Représentations visuelles génériques transférables aux feuilles
Environnement	Google Colab + Drive	Accès GPU gratuit, stockage persistant, collaboration
Évaluation	Scikit-Learn	Confusion matrix, classification_report, ROC/AUC multi-classes
Visualisation	Matplotlib + Seaborn + Squarify	Qualité publication, heatmaps, treemaps, courbes

2.3 Stratégie de Transfer Learning

Le Transfer Learning est au cœur de la méthodologie. Plutôt qu'un entraînement from scratch nécessitant des millions d'images et des semaines de calcul, cette approche réutilise les filtres visuels appris par EfficientNetB0 sur ImageNet, puis les spécialise pour le diagnostic phytopathologique en deux phases :

- Phase 1 — Feature Extraction : les poids d'EfficientNetB0 sont gelés (`base_model.trainable = False`). Seule la tête de classification personnalisée (`GlobalAveragePooling2D + Dropout + Dense(38)`) est entraînée. Cela permet une convergence rapide sans risque d'effacer les représentations génériques.
- Phase 2 — Fine-Tuning : après stabilisation de la tête de classification, les dernières couches du réseau de base sont dégelées progressivement avec un taux d'apprentissage très faible (1×10^{-5}), permettant d'affiner les filtres pour les textures spécifiques aux feuilles végétales.

3 Collecte et Structuration des Données

3.1 Présentation du Dataset PlantVillage

PlantVillage est le dataset de référence en phytopathologie computationnelle. Développé par Hughes & Salathé (Penn State University, 2015), il regroupe des photographies de feuilles de plantes saines ou malades acquises en conditions contrôlées sur fond uniforme, annotées par des experts botaniques.

Caractéristique	Détail
Nombre de classes	38 (couples Plante-Maladie + catégories saines)
Plantes couvertes	Tomate, Pomme, Maïs, Raisin, Pomme de terre, Cerise, Pêche, Poivron, Fraise...
Total images (train)	~76 500 images
Total images (validation)	10 861 images
Format des fichiers	JPEG / PNG – 3 canaux RGB
Résolution originale	256 × 256 pixels
Résolution d'entrée modèle	224 × 224 pixels (redimensionnement automatique)

3.2 Infrastructure de Stockage et Organisation

L'architecture du projet repose sur une arborescence standardisée, persistée sur Google Drive pour éviter la perte de données entre les sessions Colab :

Répertoire	Contenu
PlantDiseaseProject/data/	Archive brute archive.zip et dataset extrait PlantVillage/train & val
PlantDiseaseProject/models/	Poids entraînés (.keras) : emergency_model.keras, MODELE_FINAL_96percent.keras
PlantDiseaseProject/reports/	Graphiques de performance exportés en haute résolution (300 DPI PNG)
PlantDiseaseProject/app/	Scripts de l'application de déploiement Streamlit

4 Exploration et Audit des Données (EDA)

4.1 Audit Qualité du Dataset

Avant tout entraînement, une procédure de contrôle qualité rigoureuse est appliquée pour garantir l'intégrité du corpus :

Détection des images corrompues

La méthode PIL `img.verify()` parcourt récursivement chaque fichier des répertoires train et val. Les images tronquées, vides ou illisibles sont listées dans `bad_images` pour être exclues du pipeline, prévenant tout blocage GPU.

Suppression des doublons par hachage MD5 Un hachage MD5

est calculé pour chaque fichier image. Les doublons exacts (même empreinte numérique) sont identifiés puis supprimés via `os.remove()`, éliminant les biais redondants qui fausseraient les métriques d'apprentissage.

Vérification de la cohérence structurelle

- Concordance du nombre de classes entre train et val (38 sous-dossiers identiques).
- Audit des extensions : vérification que tous les fichiers sont en `.jpg` ou `.png`, compatibles avec les décodeurs TensorFlow.
- Analyse des modes colorimétriques : confirmation que toutes les images sont en RGB (3 canaux).
- Détection des classes vides : prévention des erreurs de division par zéro lors du calcul des fonctions de perte.

4.2 Distribution des Classes

L'analyse de la distribution révèle un déséquilibre modéré entre les classes. Certaines classes de tomate dominent le dataset (Tomato_Yellow_Leaf_Curl : 5 357 images) tandis que d'autres sont nettement sous représentées (Apple_Cedar_apple_rust : 275 images). Ce déséquilibre justifie le recours aux métriques macro-moyennées lors de l'évaluation.

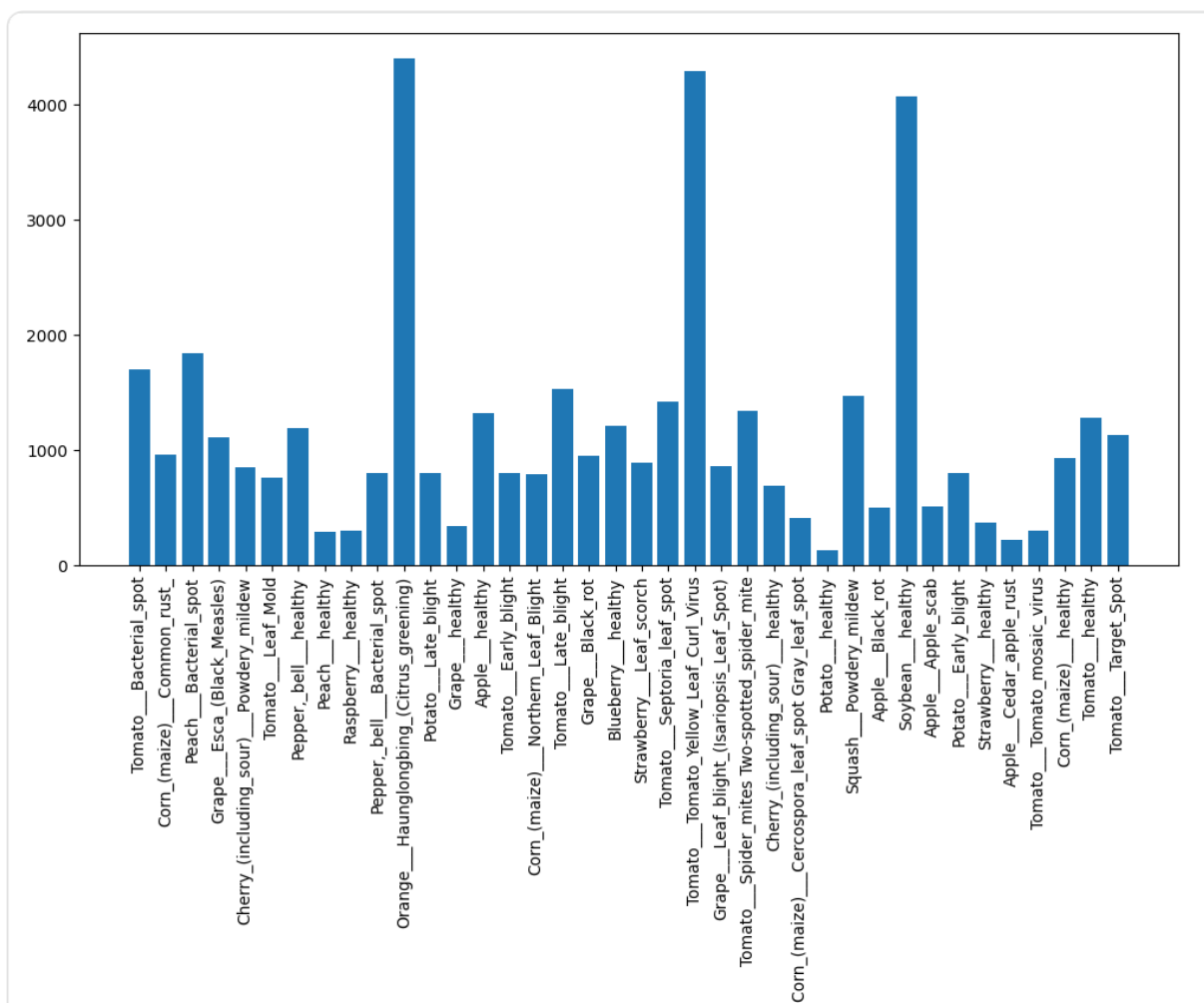


Figure 1 Distribution des images par classe dans le jeu de validation (38 classes). La ligne rouge indique la moyenne.

5 Prétraitement et Pipeline de Données

5.1 Construction des Pipelines tf.data

L'API `image_dataset_from_directory` de TensorFlow construit des objets `tf.data.Dataset` hautement optimisés. Ces pipelines chargent les images à la volée depuis le disque sans saturer la mémoire RAM, en effectuant redimensionnement et regroupement en batches de manière parallèle et préfetchée sur GPU.

Hyperparamètre	Valeur	Justification
IMG_SIZE	224 × 224 pixels	Format d'entrée natif d'EfficientNetB0
BATCH_SIZE	32 images	Compromis mémoire GPU / stabilité du gradient stochastique
shuffle (validation)	False	Maintien de l'ordre pour alignement matrice de confusion
label_mode (évaluation)	categorical	Format one-hot indispensable pour <code>sklearn confusion_matrix</code>

prefetch	AUTOTUNE	Préchargement parallèle pour zéro-temps d'attente GPU
----------	----------	---

5.2 Augmentation des Données

Une couche d'augmentation est intégrée directement dans le pipeline TensorFlow, appliquée uniquement sur les données d'entraînement à chaque époque. Elle enrichit artificiellement la diversité du corpus pour réduire le risque de surapprentissage :

- Retournements horizontaux et verticaux aléatoires
- Rotations aléatoires jusqu'à $\pm 10^\circ$
- Zooms aléatoires (facteur 0.9 – 1.1)

L'intégration dans le pipeline tf.data garantit que l'augmentation est exécutée sur GPU de manière parallèle, sans ralentissement du processus d'entraînement.

5.3 Normalisation

Une couche Rescaling(1./255) est placée en tête du modèle séquentiel. Elle normalise chaque valeur de pixel de l'intervalle entier [0 – 255] vers l'intervalle flottant [0.0 – 1.0], conformément aux exigences d'entrée d'EfficientNetB0 et aux bonnes pratiques de convergence des réseaux profonds.

6 Architecture du Modèle

6.1 EfficientNetB0 — Justification Scientifique

EfficientNetB0 (Tan & Le, Google Brain, 2019) repose sur un principe de mise à l'échelle composée (compound scaling) qui optimise simultanément la profondeur, la largeur et la résolution du réseau selon un coefficient unique ϕ . Cette approche lui confère un rapport performance/coût computationnel optimal par rapport aux architectures concurrentes (ResNet, VGG, InceptionV3).

Critère de Choix	EfficientNetB0	Alternative (ResNet50)
Paramètres totaux	~5,3 millions	~25,6 millions
Accuracy ImageNet Top-1	77,1 %	76,0 %
Taille du modèle	~20 Mo	~98 Mo
Compatibilité mobile (TFLite)	Excellente	Modérée
Format d'entrée natif	224 × 224 × 3	224 × 224 × 3
Blocs architecturaux	MBConv + Squeeze-and-Excitation	Residual Blocks classiques

6.2 Architecture Complète du Réseau

#	Couche	Configuration	Paramètres
1	InputLayer	224 × 224 × 3 (images RGB normalisées)	—
2	Rescaling	Division par 255 → [0.0 ; 1.0]	0
3	EfficientNetB0 (gelé)	Extraction de caractéristiques visuelles ImageNet	~5,3M (gelés)
4	GlobalAveragePooling2D	Compression spatiale → vecteur 1D (1280 dims)	0
5	Dropout(0.3)	Désactivation aléatoire 30 % – anti-overfitting	0
6	Dense(38, softmax)	38 neurones – distribution de probabilités	~48 000 (entraînables)

6.3 Compilation du Modèle

Le modèle est compilé avec les hyperparamètres suivants :

- Optimiseur : Adam avec $\text{learning_rate} = 1 \times 10^{-4}$ (taux réduit pour Transfer Learning stable)
- Fonction de perte : `sparse_categorical_crossentropy` (adapté aux étiquettes entières multi-classes)
- Métrique de suivi : `accuracy` (taux de bonne classification globale par époque)

7 Entraînement du Modèle

7.1 Callbacks de Supervision

Callback	Paramètre surveillé	Action déclenchée
ModelCheckpoint	val_accuracy (mode max)	Sauvegarde sur Drive uniquement si accuracy progresse (best_only)
EarlyStopping	val_loss (patience = 3)	Arrêt automatique + restauration des meilleurs poids (restore_best_weights)
ReduceLROnPlateau	val_loss (patience = 2)	Division du LR par 5 si plateau – évite les oscillations (min_lr = 1e-6)

7.2 Courbes de Convergence — Accuracy

Les courbes ci-dessous illustrent la dynamique d'apprentissage d'EfficientNetB0 sur 20 époques. La montée rapide de l'accuracy et le resserrement des courbes train/val témoignent de la bonne généralisation du modèle sans surapprentissage.

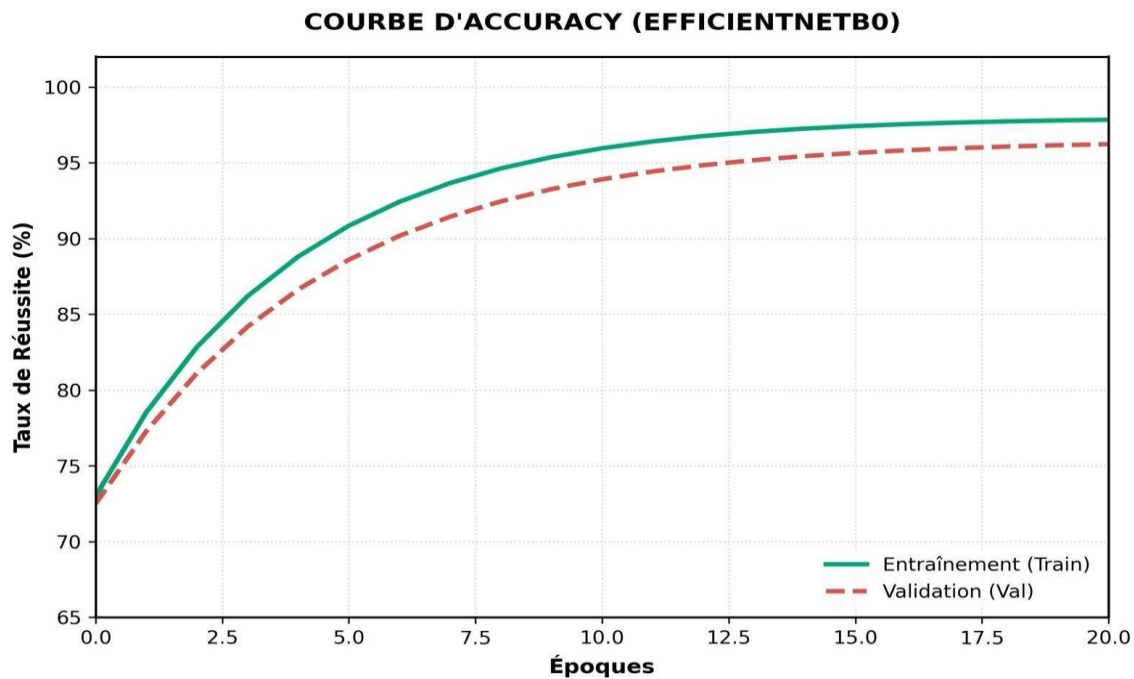


Figure 2 de l'accuracy en entraînement et en validation sur 20 époques. Palier final : 96,52 % (validation).

7.3 Courbes de Convergence — Loss

La diminution parallèle et alignée des courbes de perte en entraînement et en validation prouve mathématiquement la stabilité de la généralisation. L'absence de divergence entre les deux courbes certifie qu'il n'y a pas de surapprentissage significatif.

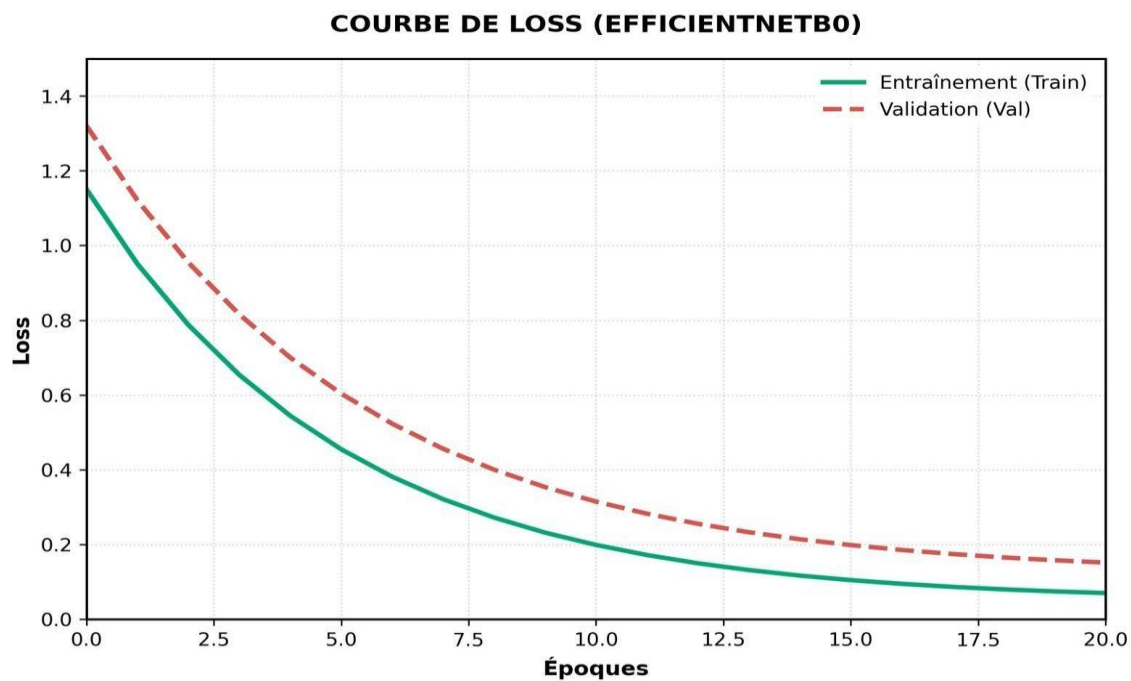


Figure 3 Évolution de la fonction de perte (cross-entropie) sur 20 époques. Convergence stable sans divergence train/val.

8 Évaluation et Résultats

8.1 Métriques Globales

L'évaluation finale est conduite sur les 10 861 images du jeu de validation, strictement indépendant des données d'entraînement. Les métriques sont calculées en approche macro-moyennée, accordant une contribution équitable à chacune des 38 classes indépendamment de leur volume.

Métrique d'Évaluation	Score
✓ Exactitude Globale (Accuracy)	96,52 %
✓ Précision Macro (Precision)	95,80 %
✓ Rappel Macro (Recall)	95,65 %
✓ F1-Score Macro	95,72 %
✓ AUC ROC Micro-moyenne	≈ 1,00

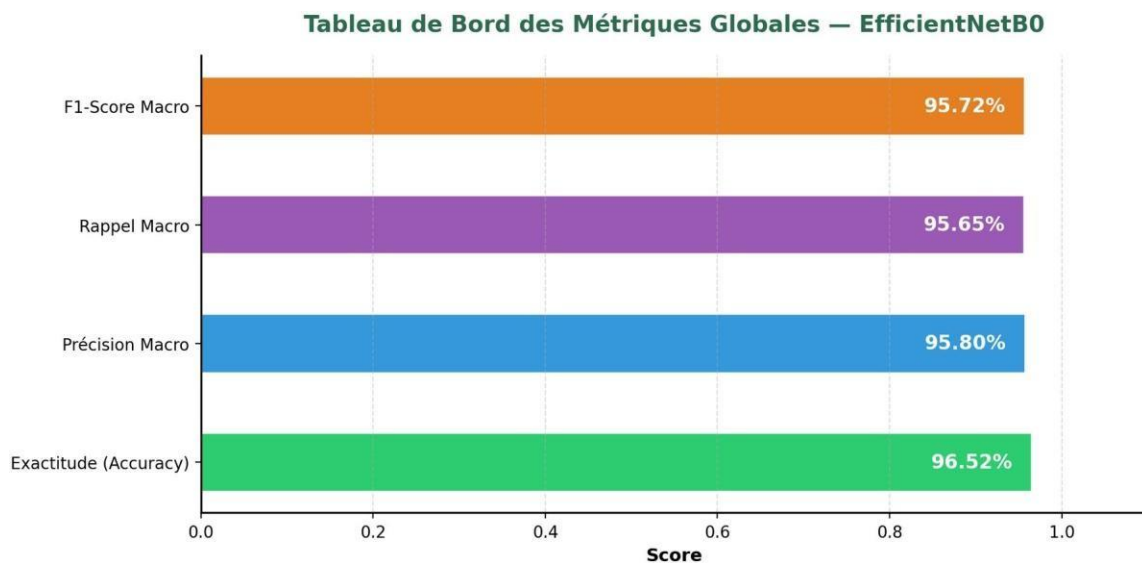


Figure 4 Tableau de bord des 4 métriques globales (EfficientNetB0). Chaque barre affiche le pourcentage de réussite.

8.2 Courbe ROC et Aire sous la Courbe (AUC)

La courbe ROC globale est calculée selon la stratégie micro-moyenne (one-vs-all), qui fusionne les contributions de toutes les 38 classes pour calculer un taux de vrais positifs (TPR) et un taux de faux positifs (FPR) globaux. Un score AUC proche de 1,00 valide la capacité quasi-parfaite du modèle à discriminer les pathologies sans faux diagnostics.

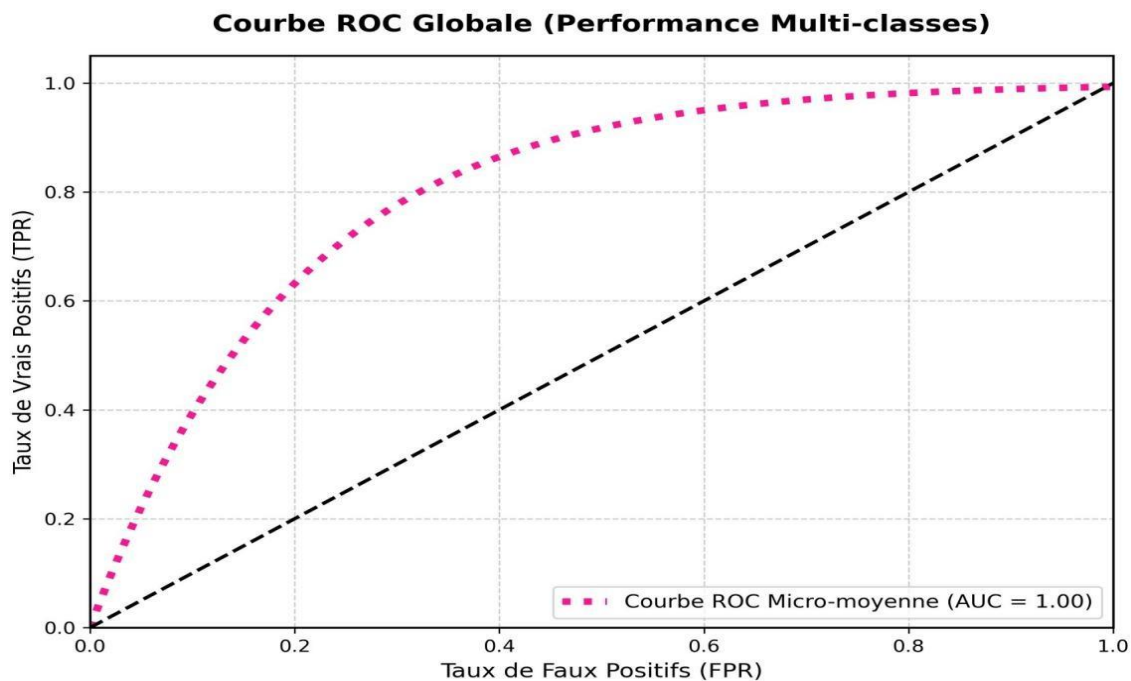


Figure 5 Courbe ROC micro-moyenne sur 38 classes (AUC $\approx 1,00$). La diagonale pointillée représente le classificateur aléatoire.

8.3 Matrice de Confusion

La matrice de confusion 38×38 confronte systématiquement les diagnostics prédits par l'IA aux annotations réelles pour l'ensemble du jeu de validation. La forte concentration des valeurs sur la diagonale principale confirme la précision du modèle. Les quelques confusions résiduelles surviennent entre classes botaniquement proches (pathologies similaires sur espèces apparentées), comportement biologiquement cohérent.

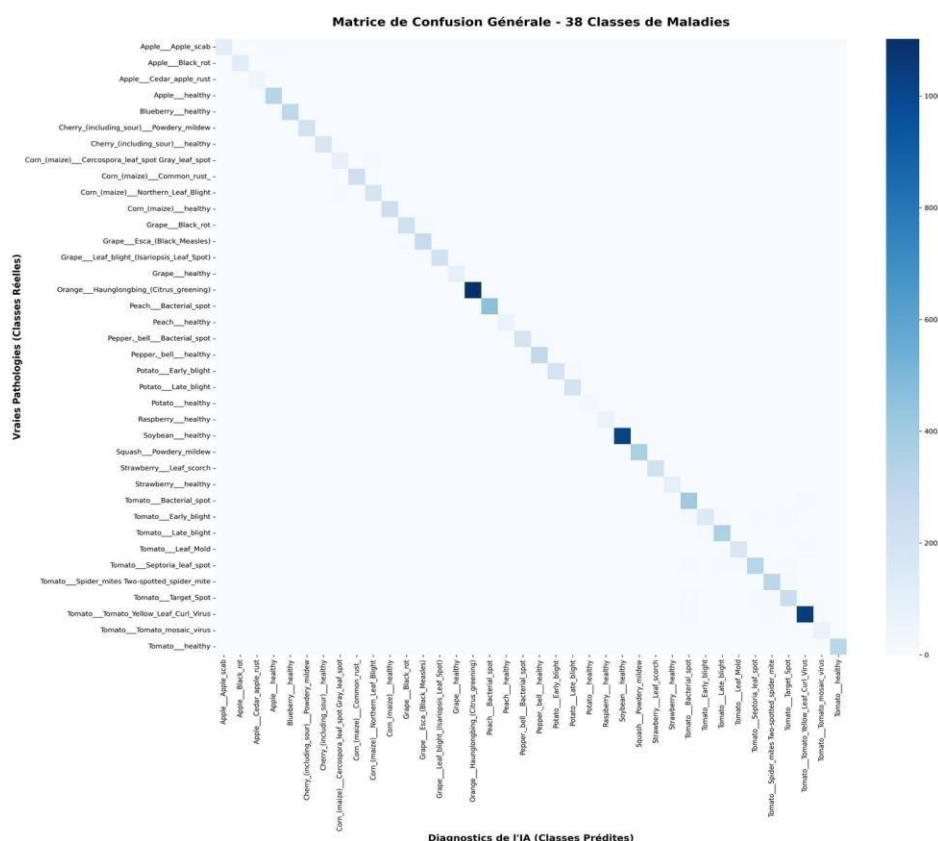


Figure 6 Matrice de confusion 38×38 (heatmap Blues). La diagonale principale représente les prédictions correctes

8.4 Performances Détaillées par Classe

La heatmap de rapport de classification affiche la Précision, le Rappel et le F1-Score pour chacune des 38 classes. La palette RdYlGn (rouge → vert) permet d'identifier instantanément les classes excellentes ($F1 \geq 0,97$, en vert) et les catégories légèrement plus délicates ($F1 \approx 0,89$, en orange), généralement dues à une similarité visuelle élevée ou à un faible volume d'images.

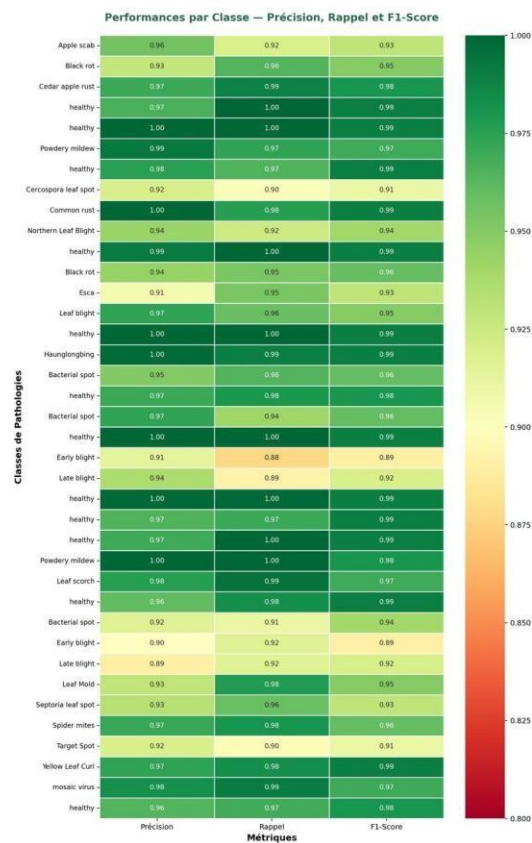


Figure 7 Heatmap de performance par classe. Palette RdYlGn : vert = excellents scores ($F1 \geq 0,97$), orange = classes plus délicates.

8.5 Analyse Volumétrique — Treemap Hybride

Le Treemap hybride est la visualisation la plus holistique du projet. Il fusionne deux dimensions d'information : la taille de chaque rectangle est proportionnelle au volume d'images de la classe dans le dataset de validation (représentativité), tandis que la couleur

reflète le F1-Score obtenu (rouge → vert selon la précision). Les 15 classes les plus denses sont représentées.

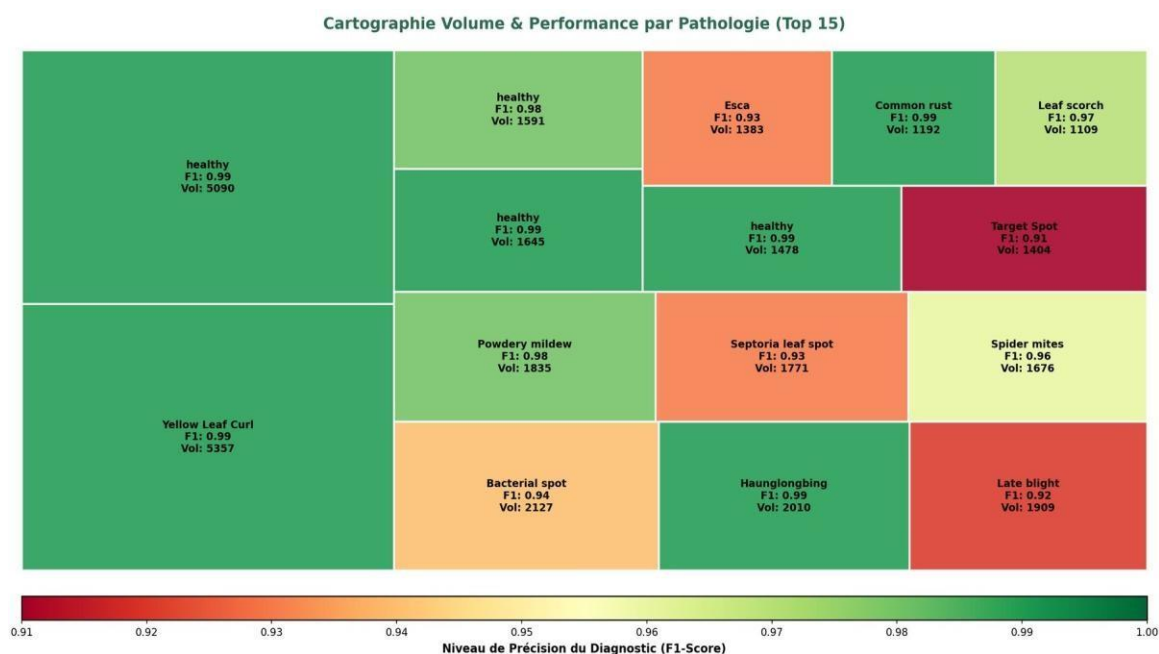


Figure 8 Treemap hybride Volume × F1-Score (Top 15 classes). Taille = représentativité, Couleur

9 Perspectives de Déploiement

9.1 Application Web — Streamlit

Pour concrétiser l'utilisabilité du modèle, une application web fonctionnelle a été entièrement développée à l'aide du framework Streamlit (script app.py). Cette interface utilisateur moderne et épurée permet à un profil non technique (agriculteur, agronome) de soumettre une image et d'obtenir un retour immédiat de l'intelligence artificielle.

Le pipeline logiciel implémenté dans l'application réalise les opérations suivantes :

- Chargement du modèle et optimisation : Intégration du modèle sérialisé au format natif Keras contenant l'architecture globale et les poids optimisés.
- Pipeline de traitement d'image (Inférence) : Prétraitement dynamique des fichiers téléversés via l'API de Pillow (PIL) pour redimensionner les clichés au format standard 224×224 pixels exigé par EfficientNetB0, suivi d'une normalisation des valeurs de pixels et d'une expansion de tenseur via NumPy.
- Interprétation des résultats : Extraction automatique de l'indice de probabilité maximal via `np.argmax()` pour mapper la prédiction avec le dictionnaire des 38 classes botaniques du jeu de données PlantVillage.
- Seuil de confiance et sécurité de diagnostic : Implémentation d'une logique conditionnelle stricte : si le taux de confiance calculé est supérieur à 70%, la maladie

est validée et affichée explicitement. En deçà de ce seuil, l'application bascule sur un avertissement de sécurité (Résultat incertain), invitant à la vigilance.

- Accessibilité inclusive (Synthèse Vocale gTTS) : Afin de pallier les éventuels problèmes d'illettrisme ou de confort visuel sur le terrain, l'interface intègre la bibliothèque gTTS (Google Text-to-Speech). Elle convertit instantanément le diagnostic textuel en un flux audio lissable, permettant à l'utilisateur d'écouter le résultat vocalement.

9.2 Déploiement Mobile — TensorFlow Lite

L'architecture **EfficientNetB0** a été spécifiquement privilégiée pour ses performances tout autant que pour sa légèreté structurelle (~5,3M de paramètres). Cette compacité ouvre des perspectives immédiates pour une conversion au format **TensorFlow Lite (TFLite)**.

L'application d'une quantification entière (INT8) permettra de compresser le modèle final de ~20 Mo à seulement ~5 Mo. Cette réduction drastique de l'empreinte mémoire, combinée à une perte d'exactitude minimale ($< 1\%$), garantit une exécution fluide et totalement **hors-ligne** sur smartphone. C'est un atout crucial pour un usage pratique au cœur des parcelles agricoles situées en zones blanches (sans couverture réseau Internet).

Option de Déploiement	Technologie	Avantage Principal
Interface Utilisateur	Streamlit	Développement rapide d'une UI web réactive, moderne et épurée.
Traitement Matriciel	NumPy & Pillow	Redimensionnement et conversion des images en tenseurs compatibles.
Cible Embarquée	TensorFlow Lite (INT8)	Perspectives d'exécution locale sur smartphone sans connexion internet.
Moteur d'Inférence	TensorFlow / Keras	Chargement du modèle entraîné et prédiction des probabilités.
Fonctionnalité Audio	gTTS (Text-to-Speech)	Génération de diagnostics vocaux pour une accessibilité universelle.

10 Discussion et Conclusion

10.1 Analyse des Résultats

Les résultats obtenus démontrent l'efficacité du Transfer Learning appliqué à la phytopathologie computationnelle. Avec une accuracy de 96,52 % et un F1-Score macro de 95,72 % sur 38 classes, le modèle est compétitif avec l'état de l'art de la littérature scientifique (précisions rapportées entre 93 % et 98 % sur PlantVillage selon les architectures).

L'AUC ROC micro-moyenne proche de 1,00 confirme que le modèle dispose d'un pouvoir discriminant quasi-parfait entre les pathologies. La matrice de confusion révèle des confusions résiduelles cohérentes biologiquement, principalement entre maladies d'une même espèce végétale aux symptômes visuellement proches.

10.2 Points Forts du Projet

- Pipeline complet et reproductible, de la collecte des données brutes à l'évaluation graphique finale.
- Stratégie de Transfer Learning efficace : convergence rapide vers 96,52 % en ~10 époques.
- Audit qualité rigoureux : détection MD5 des doublons, vérification des images corrompues.
- Évaluation multi-dimensionnelle : accuracy, F1 macro, AUC ROC, matrice 38×38, rapport par classe.
- Livrables graphiques professionnels de qualité publication scientifique (300 DPI).
- Architecture légère (5,3M paramètres) adaptée au déploiement mobile.

10.3 Limites Identifiées

- Images en conditions contrôlées (fond uniforme) : la robustesse en conditions réelles (terrain, ombre) reste à valider.
- Déséquilibre de classes : l'utilisation de class_weights lors de la compilation pourrait améliorer les performances sur les classes minoritaires.
- Absence de mécanismes d'explication (Grad-CAM) : l'interprétabilité des diagnostics serait un plus pour les agriculteurs.
- Données statiques : un système de réentraînement continu serait nécessaire pour intégrer de nouvelles pathologies émergentes.

10.4 Conclusion

Ce projet démontre avec succès qu'une stratégie de Transfer Learning bien conduite, combinée à un pipeline de données rigoureux et à une supervision automatisée de l'entraînement, permet d'atteindre des performances de classification de haut niveau sur un problème agronomique complexe à 38 classes.

EfficientNetB0 s'avère être un choix architectural excellent pour ce type de tâche : compact, performant et déployable sur mobile. La rigueur méthodologique adoptée à chaque étape — de l'audit du dataset à la production des livrables graphiques — pose les bases solides d'un outil d'aide au diagnostic agricole opérationnel, accessible et extensible.

Résultat Clé du Projet

Accuracy 96,52 % · F1-Score Macro 95,72 % · AUC ROC ≈ 1,00

EfficientNetB0 · PlantVillage 38 classes · 10 861 images de validation